Letting a Robot Train Itself: What Helps, What Doesn't, and What Only Looks Like It Helps

Anonymous CVPR submission

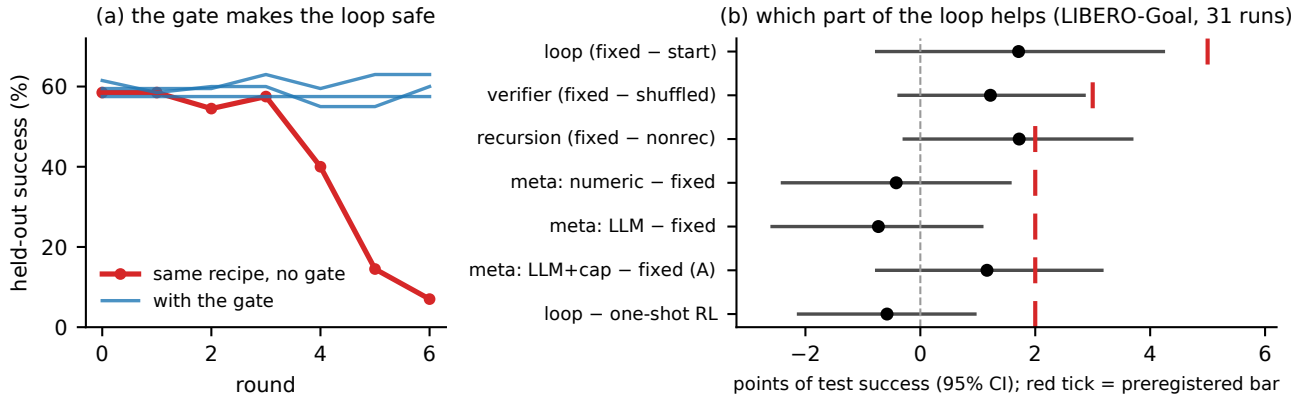


Figure 1. **Taking a self-improvement loop apart.** (a) The gate is what makes self-training safe: the same behaviour-cloning recipe falls from 58.5% to 5.5% success when every new version is accepted, and never drops below its start when each version has to beat its parent on fresh, paired placements. (b) Switching off one ingredient at a time (31 LIBERO-Goal runs, official test kept sealed, finals averaged over three evaluations): every ingredient's estimate is positive except the meta level's, and none reaches the bar we set in advance (red ticks). Recursion and the verifier matter most; a language model that edits the training recipe adds nothing once the recipe works.

Abstract

A robot policy can improve without new demonstrations: it attempts tasks, a verifier marks which attempts succeeded, it trains on those, and the improved policy practises again. Loops of this kind are reported to work for language models and, more recently, for vision-language-action (VLA) policies, but the reports rarely say which part of the loop produced the gain. The loop bundles several ideas: the verifier, the recursion (the latest policy generates the next practice), a gate that rejects versions that do not beat their parent, and sometimes a "meta level" that rewrites the training recipe. Any of them could be doing the work; so could ordinary reinforcement learning with the same compute. We built one framework in which each part is a switch, committed every comparison, bar and statistic to a public preregistration before running it, kept the official benchmark locked until all decisions were made, and averaged every final score over repeated evaluations. On LIBERO-Goal with a 7B OpenVLA-OFT policy (31 runs, two GPU platforms) the gate is what makes self-training safe: a recipe that collapses from 58.5% to 5.5% success when every version is accepted never drops below its start under the gate. Beyond safety the gains are small. The loop adds one to two points; recursion contributes $+1.7$ points (95% CI $[-0.3, +3.7]$) and the verifier $+1.2$ $[-0.4, +2.9]$ at six rounds, neither clearing its bar, and one round of plain RL at equal compute does as well. At twelve rounds recursion separates by $+7.8$ $[+5.2, +10.5]$, because the loop without it drifts below its start through a chain of gate-accepted versions while the recursive loop holds its level. A meta level, whether a numeric search or a language model that reads the training records, adds nothing once the recipe works (within ± 1.2 points on two studies), even though its diagnoses are correct: every recipe in the space reaches the same plateau. It can matter only when the recipe is broken, which a sealed self-repair study tests. In a second domain, predicting program outputs with four language models, the loop gains $+2.6$ to $+5.8$ points and recursion becomes significant at eight rounds. Finally, a published VLA-RL score owes $+16.8$ points $[+9.8, +23.4]$ to training on the benchmark's own test states. Code, preregistrations and all run records are

released.

1. Introduction

Collecting demonstrations is the expensive part of training a robot policy. An attractive alternative is to let the policy teach itself: give it a task, let it try, keep the attempts that worked, train on them, and let the improved policy try again. For language models this recipe is called self-improvement and has several working versions [2, 5, 23–25]; reinforcement learning with a verifiable reward is its standard form for reasoning models [3, 20]. Robot learning has started to follow: a vision-language-action (VLA) policy can be improved by RL in simulation from a success flag alone [13, 15]. The most ambitious version, often called recursive self-improvement, goes one step further and lets the system also revise the procedure by which it improves.

Reading these papers, we kept asking two plain questions. First, *which part is doing the work?* A self-improvement loop contains at least four ideas that could each be responsible for a gain: the verifier that labels attempts as successes, the recursion that lets the newest policy generate the next practice, a gate that keeps only versions that beat their parent, and a meta level that edits the recipe. It also contains ordinary RL as a special case, so a positive result does not by itself show that any of the four mattered. Second, *is the gain real?* Benchmarks such as LIBERO [14] test on a fixed list of initial object placements, and nothing stops a method from training on that list. We show that a widely used VLA-RL checkpoint does, and that about a fifth of its score comes from it (Sec. 5.5).

This paper answers both questions by experiment. We built a framework (Sec. 3) in which every ingredient of the loop is a switch inside one engine: the verifier can be real or replaced by shuffled labels; practice can come from the latest policy or from the frozen initial one; the meta level can be off, a numeric search, or a language model that reads the round records and edits the recipe inside a declared space; and the whole budget can instead be spent on one long round of plain RL. Flipping one switch at a time attributes a difference to one ingredient.

Self-improvement results are easy to inflate: the loop selects versions, evaluation is noisy, and the experimenter sees the intermediate numbers. So we fixed the rules before playing (Sec. 4). Every comparison, bar and statistic was written down and committed to a public repository before the corresponding runs started; every later change is a dated amendment that says what had been seen; the official test was never used for training, sampling or selection and was opened once per study; and every final score is the average of three evaluations, because a single one carries about ± 1.5 points of noise.

What we found. Figure 1 summarises the results.

1. **The gate makes self-training safe, with a condition.** Across 31 six-round runs with an OpenVLA-OFT policy on LIBERO-Goal, no gated run ends below its start beyond evaluation noise, while the same recipe collapses from 58.5% to 5.5% when every version is accepted; over twelve rounds the protection holds only when practice is recursive (Sec. 5.1, Sec. 5.2).
2. **Beyond safety the gains are small, and no single ingredient clears its bar at six rounds.** The loop gains one to two points. Recursion contributes $+1.7$ points $[-0.3, +3.7]$ and the verifier $+1.2$ $[-0.4, +2.9]$; one round of plain RL with the same budget does as well. At twelve rounds recursion separates by $+7.8$ $[+5.2, +10.5]$, but because the loop without it drifts below its start while the recursive loop holds its level; plain RL still ties (Sec. 5.2).
3. **A meta level does not help a recipe that already works, and we can say why.** Four variants, including a language model allowed to edit nine components of the training harness, land within ± 1.2 points of a fixed recipe; a second, separately registered study gives -0.3 and $+0.3$ points on two platforms. The model's diagnoses are correct, but every recipe in the space reaches the same plateau (Sec. 5.3).
4. **A meta level becomes distinguishable from RL only when the recipe fails.** Starting from a recipe that collapses without the gate, the language model names the cause unprompted and the gate keeps its first repair on three of four seeds; the official result of this self-repair study is sealed until the study ends (Sec. 5.3).
5. **In a second domain the loop gains more, and recursion pays only at long horizon.** Language models trained to predict the output of a program gain $+2.6$ to $+5.8$ points from verified practice; recursion is worth $+2.8$ $[+1.2, +4.5]$ after eight rounds and nothing after four (Sec. 5.4).
6. **A published VLA-RL score owes about a fifth of itself to the test states** ($+16.8$ points $[+9.8, +23.4]$; Sec. 5.5).

Contributions. (i) A self-improvement framework for VLA policies in which every control is a switch inside one engine, with supervised fine-tuning, behaviour cloning and GRPO as interchangeable update rules, and a language-model meta level that edits a declared recipe space under an attribution guard. (ii) A preregistered, sealed-test protocol for self-improvement claims, with every amendment public. (iii) The first controlled decomposition of such a loop on a VLA benchmark, including its negative results and the mechanism behind them. (iv) A positive recursion result in a second domain at longer horizon. (v) A measurement of train-on-test inflation in a published VLA-RL result. Code, preregistrations and all run records are released.

2. Related work

Self-improvement with verifiers. STaR [24], ReST [5] and ReST-EM [21] fine-tune a language model on its own verified rationales; SPIN [2] and self-rewarding models [23] close the loop with the model as its own judge; Absolute Zero [25] lets the model write its own tasks and uses a code executor as the verifier. RLVR with group-relative policy optimisation [3, 20] is the dominant update rule. These works report that the loop helps. We ask which part of it helps, with every part switchable and every comparison registered in advance. The closest analyses are the iterated on-policy convergence results of [12], whose predicted shape (early saturation of a learner restricted to samples from the initial policy) matches our code-domain recursion result.

Meta levels that edit the procedure. LLM-driven search over programs and training recipes has produced strong results when the search space is large and evaluation is cheap [6, 16, 19]. In our setting evaluation is expensive (hundreds of simulator episodes per candidate) and noisy, and the space is a few dozen knobs and harness components. Under these conditions, we find, a language-model meta level that diagnoses correctly still cannot beat a fixed recipe that already works, and we isolate the one case where it could.

RL for VLA policies. OpenVLA [10] and OpenVLA-OFT [11] are the policies we improve; SimpleVLA-RL [13], RL4VLA [15] and related work [17] apply on-policy RL to such policies in simulation with a binary success reward. We use the SimpleVLA-RL training stack as one update operator and its released checkpoints as starting points. Our train-on-test measurement (Sec. 5.5) concerns the evaluation practice of this line of work rather than its methods.

Preregistration. Preregistered designs are standard in the experimental sciences [18] and have been proposed for machine learning [1]. We follow the practice literally: every bar, statistic and read-once rule was committed to a public repository before the corresponding runs, and every change is a dated amendment that states what had been seen.

3. Framework

Figure 2 shows the framework. In plain terms: a policy practises on freshly sampled scenes, a verifier marks each attempt, the verified attempts update the policy, and a gate decides whether the updated policy replaces the current one. Above this sits an optional meta level that may edit the training recipe. The rest of this section states that picture precisely, together with the switches the experiments flip.

3.1. Setting and notation

A policy π_θ acts in scenes $s \sim \mathcal{S}$ (object placements of a task, or programs with inputs); an attempt $\tau \sim \pi_\theta(\cdot \mid s)$ is a trajectory, and an external oracle $v(s, \tau) \in \{0, 1\}$ the policy cannot influence labels it (the simulator's success flag; a sandboxed interpreter). A *program* P is a JSON document with four sections—`curriculum` (which scenes to practise, how many, how allocated across tasks), `solver` (attempts per scene K, temperature T), `curation` (which attempts become training items) and `update` (the operator and its hyperparameters)—and $\mathcal{P}$ is a declared edit space of allowed values. A *version* is a pair (θ, P). The framework keeps an archive of versions; the one it currently trusts is the *retained* version $(\theta^\star, P^\star)$.

3.2. The loop (Algorithm 1)

One *inner step* runs program P from policy θ: propose scenes, attempt each K times, verify, curate, update. Curation `mixed_groups` keeps every attempt of scenes whose success rate lies strictly inside a band (used by GRPO [20]: the group-centred advantage is zero when all attempts agree); `verified` keeps successful attempts only (behaviour cloning, BC). The update is a LoRA step on θ. A *round* is three inner steps from the same parent, producing a *candidate* θ'. Because the next round starts from the retained version, practice is always generated by the latest policy: this is the *data recursion*, and the control `nonrec` breaks exactly it by proposing and attempting with the frozen initial policy θ_0 while still updating from $\theta^\star$.

3.3. Gate, audit and sealed test

Three disjoint evaluation sets are drawn per run from seeds that never overlap the official benchmark states. The *gate* (20 placements per task, 200 episodes, greedy decoding, environment re-seeded per episode so that versions are compared on identical placements) decides: a candidate is kept iff its gate success is at least the parent's on the same placements. In the *fresh-paired* mode used in every main run a new gate set is drawn every round and the parent is re-evaluated on it, so acceptance is always a paired comparison on placements never seen before, and a version cannot overfit a fixed gate. The *audit* (200 more) is recorded but never decides anything; it exposes gate overfitting. The *test* is the official benchmark (50 initial states per task, 500 episodes) and is *sealed*: no process in the loop can read it, and it is opened once, after every decision in every run of a study has been made.

This is the *selection recursion*: the system's own verification result decides its next version, and the kept version decides who generates the next practice.

Algorithm 1 Gated recursive self-improvement with a meta level

Require: initial policy θ_0, seed program P_0, oracle v, gate set size n_g, rounds R, candidates per round c

$(\theta^\star, P^\star) \leftarrow (\theta_0, P_0)$; evaluate $\theta^\star$ on fresh gate and audit sets
for $r = 1 \dots R$ **do**
  draw a fresh gate set G_r of n_g paired placements; re-evaluate $\theta^\star$ on G_r
  $\{P_1, \dots, P_c\} \leftarrow \{P^\star\} \cup$ PROPOSE(round records, $\mathcal{P}$) ▷ meta level: fixed / numeric / LLM
  for each candidate program P_i **do**
    $\theta_i \leftarrow \theta^\star$
    for inner step $= 1 \dots 3$ **do**
      scenes $\leftarrow$ PROPOSE$(P_i, \theta_{\text{gen}})$ with $\theta_{\text{gen}} = \theta_i$ (recursive) or θ_0 (`nonrec`)
      attempts $\leftarrow K$ samples per scene at temperature T; labels $\leftarrow v(\cdot)$ (or permuted: `shuffled`)
      items $\leftarrow$ CURATE(attempts, P_i); $\theta_i \leftarrow$ UPDATE(θ_i, items, P_i)
    end for
    $g_i \leftarrow$ success of θ_i on G_r (greedy); record audit, never use it
  end for
  $i^\star \leftarrow \arg\max_i g_i$; **if** $g_{i^\star} \geq g(\theta^\star, G_r)$ **then** $(\theta^\star, P^\star) \leftarrow (\theta_{i^\star}, P_{i^\star})$ ▷ gate
end for
read the sealed test of $\theta^\star$ once, after every run of the study has ended

3.4. Meta level

The meta level versions the program and lets it be improved under the same gate. Each round the incumbent $P^\star$ and $c-1$ proposed programs are each trained for real from the same parent, and the gate picks the version to keep; a program is credited only when its child wins. Three proposers:

- `fixed`: no change (the meta level is off).
- `numeric`: random mutation of declared knobs, biased toward knobs whose past changes raised the gate.
- `llm`: a language model (Claude Sonnet) reads the current program, the edit space with a neutral description of what each option computes, and, for every past candidate, its program diff, gate score, acceptance and per-step statistics (sampled success, share of mixed scenes, item counts, first/last training loss). It must write a diagnosis, then return one new program.

The edit space has two sizes. The *knob* space has ten hyperparameters (scenes, allocation, samples, temperature, curation rule and budget, operator, learning rate, steps, batch). The *harness* space adds nine components of the procedure itself: the ratio computation of the policy gradient (legacy or matched), the loss aggregation (token or episode), the BC likelihood temperature (unit or rollout), advantage normalisation, the curation band, the per-scene cap, the allocation floor and the two clip bounds. Each corresponds to a change that was made by hand during development after diagnosing round records; the LLM is never told which option is better, so rediscovering them is a test of the meta level. This is the *procedure recursion*.

Table 1. **Controls.** Each arm flips one switch of Algorithm 1.

arm	change in Algorithm 1	isolates
`fixed`	none (proposer off)	gated recursive loop
`nonrec`	$\theta_{\text{gen}} = \theta_0$	data recursion
`shuffled`	labels permuted per scene	the verifier
`onebig`	R=1, $3R$ inner steps	gate + iteration vs one-shot RL
`full`	numeric proposer, c=2	meta level (equal compute)
`llm`∗	LLM proposer, c=2	meta level (equal compute)

∗ `llm`: knob space; `llm_harness`: harness space; `llm_harness_cap`: harness space under the compute cap.

Attribution guard. The meta level never sees the audit or test, the policy weights, the scenes or the episodes; it only chooses values inside $\mathcal{P}$, every proposal passes the same validator as a numeric mutation, invalid proposals and failed calls fall back to numeric search and are counted, and a *compute cap* scales any proposal so that no candidate does more rollouts or optimiser work per step than the starting program. All practice data still comes from the policy's own attempts and all verdicts from the oracle; the LLM is a source of procedure choices, not of data.

3.5. Every control is a switch

Table 1 lists the arms used in the experiments. Each differs from `fixed` in exactly one line of Algorithm 1, so a difference between an arm and `fixed` is attributable to that line; `full` and the `llm` arms train the same number of candidates and so cost the same.

4. Experimental setup

4.1. Benchmarks, policies and starting points

LIBERO / OpenVLA-OFT. LIBERO-Goal [14] has ten goal-conditioned tasks in one scene; LIBERO-10 ten long-horizon tasks. The policy is OpenVLA-OFT [11] (7B, action chunks of 8), started from a supervised fine-tune on *one* demonstration per task [13] so that there is room to improve (about 59% on LIBERO-Goal, 19% on LIBERO-10). Rollouts and GRPO/BC training use the SimpleVLA-RL stack with LoRA (r=32). The seed program—chosen from sealed pilots and frozen by amendment before any main run—is GRPO with learning rate 3×10^{-5}, matched-ratio log-probabilities, episode-balanced loss, 8 samples per

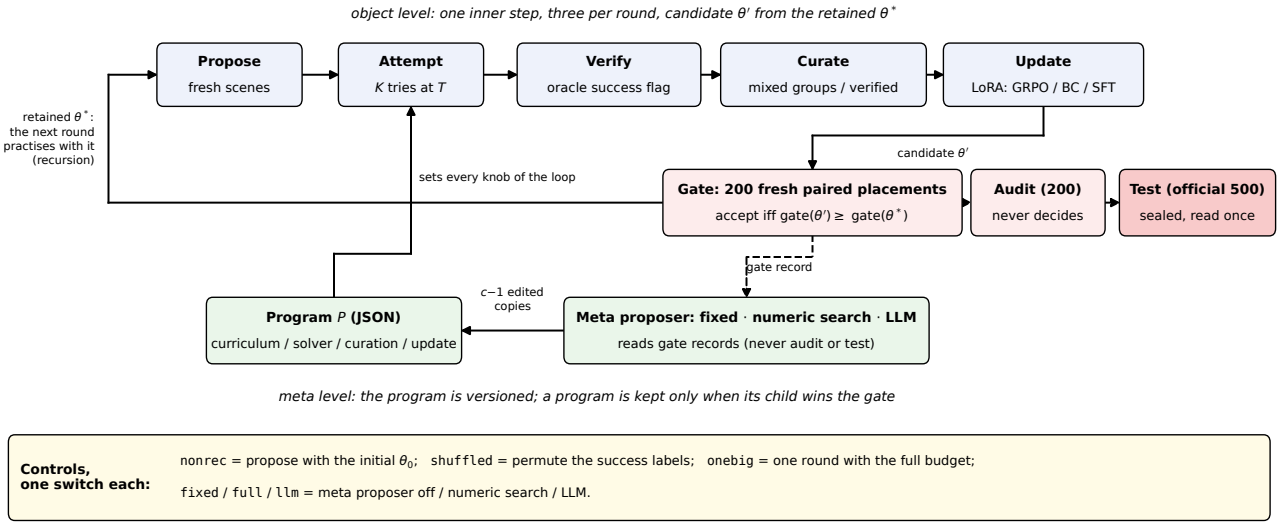


Figure 2. **The framework.** One engine, every ingredient a switch. The object loop practises on fresh scenes with the latest policy; the gate keeps a candidate only if it does not regress on paired held-out placements; the meta level versions the training program and may let a numeric search or an LLM edit it inside a declared space, seeing only gate records. Controls flip one switch each.

scene at temperature 1.5, 160 scenes per inner step, 200 optimiser steps of batch 64; 6 rounds of 3 inner steps; fresh paired gates. **Code world models.** The model predicts the output of a Python program on an input (the CRUXEval-O format [4]); it writes its own programs (3-shot from a pool that starts with 16 hand-written seeds and grows with admitted programs; a program is admitted only if it is deterministic, literal-valued and not a near-duplicate of any held-out problem), the sandbox executes them, and an attempt is verified iff its answer equals the interpreter's. Qwen2.5-Coder 1.5B and 7B [7], Qwen3-8B [22] and Granite-3.3-8B [8] are trained with LoRA SFT on at most two verified traces per case; the gate is a 400-program validation set proposed by the *initial* model; tests are CRUXEval-O/I and LiveCodeBench-execution [9].

4.2. Arms and baselines

The LIBERO-Goal main study runs the eight arms of Sec. 3.5 with three seeds each (a fourth seed was added to the `nonrec`, `shuffled`, `onebig` and `fixed` arms by amendment before any of their runs ended), 31 runs in all, on two GPU platforms: a shared 8×A800 node and a 16-GPU cloud pool with its own evaluation stack. The stacks are not bit-identical, so every comparison is paired within a platform and the platforms serve as replicates. `onebig` is the plain-RL baseline: the whole six-round rollout and update budget in one round of 18 on-policy inner steps from the initial policy, with the same sampling and update code; it is read both through the gate and as its ungated single candidate. The code-domain study runs `fixed`, `shuffled`, `nonrec`, `full` and `onebig` for four rounds, plus an eight-round `fixed` vs `nonrec` study on two models.

4.3. Preregistered protocol

Self-improvement claims are easy to inflate: the loop selects versions, evaluation is noisy, and the experimenter sees the intermediate numbers. We therefore fixed the protocol before the runs and release the preregistration documents (27 dated amendments for the VLA study) with the code.

Sealing. The official LIBERO test states are never used for training, sampling, gating or program selection, and no test file is opened before a study's read-once. Only two sealed pilots per model were allowed before the seed program was frozen.

Finals and reference. A single 500-episode evaluation is itself noisy: the *same* initial policy evaluated in different runs scored between 57.2% and 60.0% (sd ≈ 0.7), and about 40% of episodes flip between nearby policies. Each run's final is therefore the per-episode mean of three evaluations of its retained policy (the run's own test and two replicate evaluations from the same plan and source tree), and the initial reference is the per-episode mean of every in-run evaluation of the initial policy on that platform (36 evaluations on the cloud pool, 21 on the A800). A noise check—the same statistic between two halves of the initial-policy evaluations—is reported with every verdict.

Claims and bars. Seed-averaged per-episode paired bootstrap (10,000 draws), 95% CI. "CI > 0" means the interval excludes zero.

claim	comparison	bar (points)
R1 improvement	`fixed`, `full` − reference	$\geq +5$, CI > 0, 2/3 seeds > 0
R2 verifier	`fixed` − `shuffled`	$\geq +3$, CI > 0
R3 recursion	`fixed` − `nonrec`	$\geq +2$, CI > 0
R4 meta level	each meta arm − `fixed`	$\geq +2$, CI > 0
R5 vs one-shot RL	best gated arm − `onebig`	$\geq +2$, CI > 0
M2-A meta v2	`llm` − identical-recipe replica	$\geq +3$, CI > 0
S1 self-repair	`llm_harness` − `fixed` (failing start)	$\geq +3$, CI > 0

Negative readings are reported as such; the reading scripts were committed before each read and their outputs are released.

5. Results

We organise the results around five questions. Every number below comes from a preregistered read of a sealed test unless it is marked *gate-level*.

5.1. RQ1: Does the gated loop improve a VLA policy, and is it safe?

Finding 1: safe, and modestly effective. Figure 3 shows all 31 runs. Every gated run ends within noise of its start or above it (finals 57.1–62.2% against a reference of 58.4–58.9%; the two runs below their reference are within -1.3 and -1.7 points). The typical gain is one to two points and the best single seed gains $+3.3$ $[+0.1, +6.6]$. The registered R1 claim (Tab. 2) comes to $+1.71$ $[-0.76, +4.23]$ for `fixed` and $+1.28$ for `full`, below the $+5$ bar. The safety is the gate's doing, not the recipe's. In always-accept pilot curves on LIBERO-10 every recipe stayed within ± 3 points of its start for 18 on-policy steps, but the behaviour-cloning recipe of Sec. 5.3 collapsed from 58.5% to 5.5% when its versions were accepted unconditionally (Fig. 1a); under fresh paired gates the same recipes never fall below their start on the sealed test. Two qualifications. The price is noise: a 200-episode paired gate has a standard deviation of about 3.4 points, so it accepts and rejects many candidates by chance. And the protection is not unconditional: at twelve rounds the arm whose practice comes from the frozen initial policy drifts 4–11 points below its start through a chain of accepted versions (Sec. 5.2, Finding 2b), so the finding is that the gate keeps a loop with recursive practice safe at this budget, not that it is a ratchet. Acceptance rates are in the released records.

5.2. RQ2: Which part of the loop helps?

Finding 2: recursion and the verifier are the largest ingredients, neither clears its bar, and plain RL ties. Recursion (R3) comes to $+1.72$ $[-0.28, +3.68]$ against a $+2$ bar. A preliminary read on single evaluations had passed it at $+2.40$; the pass did not survive averaging over replicates, which is exactly what the protocol is for. The verifier (R2) is worth $+1.22$; without it the loop ends where it started (57.8–60.7%). One round of plain RL with the same budget matches or beats the loop (R5 -0.58; on seed 0 the one-shot policy scores 62.7 against the gated loop's 57.2). Figure 4 shows what happens at the gate: `nonrec` stops climbing after round 3 while `fixed` keeps going, the same shape we see in the code domain, where the separation becomes significant at eight rounds (RQ4).

Finding 2b: at twelve rounds recursion separates by $+7.8$ points, because the non-recursive loop degrades. A registered extension (amendment 27; three seeds each of `fixed12`, `nonrec12` and a 36-step `onebig12`, same rule for finals) gives R3-long $= +7.84$ $[+5.22, +10.49]$ against its $+2$ bar, positive on every seed ($+14.4$, $+4.9$, $+4.2$). The mechanism is not the one we predicted. `fixed12` ends where `fixed` ended at six rounds (-0.09 $[-1.98, +1.78]$ against the same seeds' 6-round finals; $+1.60$ $[-0.88, +4.13]$ over the reference, below the $+5$ bar), while `nonrec12` ends 4–11 points *below its own start* on every seed (-6.44 $[-9.00, -3.96]$ against its 6-round finals), although every one of its versions was accepted by a fresh paired gate. With one candidate per round, acceptance on $\geq$, and candidates that are truly a little worse than their parent (their practice is generated by a policy the learner has drifted away from), noise-driven acceptances accumulate. Recursion is what keeps the gated loop from drifting down at this horizon; it does not lift it. Against plain RL the picture is unchanged: R5-long $= -0.38$ $[-2.42, +1.64]$, and `onebig12`'s single candidate was kept by the gate on all three seeds.

5.3. RQ3: Does a meta level help, and when could it?

Finding 3: no meta level beats a working fixed recipe, on two studies and two platforms. All four R4 arms land within ± 1.2 points of `fixed` (Tab. 2). One could object that these arms give the LLM little to work with: each round compares only the newest edit against the incumbent, and the LLM never sees per-task results. So we registered a second study ("v2") in which the LLM sees per-task gate rates with their noise, each candidate starts from its parent's statistics, and the comparison is between an `llm` arm and a *replica* arm that trains the identical recipe at the same compute, from two starting policies. The primary bar fails on both platforms: -0.27 $[-2.53, +2.00]$ (cloud, seeds 1–3) and $+0.30$ $[-1.60, +2.15]$ (A800, seeds 0–3). Pooled over seven seed pairs, any true effect is within about ± 2 points.

Finding 4: the null has a mechanism. Over the v2 study the LLM's proposals scored -1.2 points against the incumbent at the gate (n=30), while replicas of the identical recipe scored $+0.6$ (sd 3.4, n=36): there is no edit effect for a finer gate to resolve. The LLM's diagnoses, logged verbatim, read the loop correctly. They point to the

Table 2. **Registered claims on LIBERO-Goal** (official 500-episode test, 3-evaluation finals, pooled initial reference, seed-averaged paired bootstrap). No bar is met. Platform Y = cloud pool, A = A800.

claim	comparison	points	95% CI	bar	seeds
R1	`fixed` − ref. (Y)	+1.71	[−0.76, +4.23]	+5	1–3
R1	`full` − ref. (Y)	+1.28	[−1.37, +3.99]	+5	1–3
R2	`fixed` − `shuffled` (Y)	+1.22	[−0.37, +2.85]	+3	0–3
R3	`fixed` − `nonrec` (Y)	+1.72	[−0.28, +3.68]	+2	0–3
R4	`full` − `fixed` (Y)	−0.42	[−2.40, +1.56]	+2	1–3
R4	`llm_harness` − `fixed` (Y)	−0.73	[−2.58, +1.07]	+2	1–3
R4	`llm` − `fixed` (A)	+1.04	[−0.84, +3.00]	+2	1–3
R4	`llm_harness_cap` − `fixed` (A)	+1.16	[−0.76, +3.16]	+2	1–3
R5	best gated − `onebig` (Y)	−0.58	[−2.12, +0.95]	+2	0–3
noise	half vs. half of the n0 evaluations (Y / A)	+0.64 / +0.54	[−0.07, +1.36] / [−0.24, +1.35]		

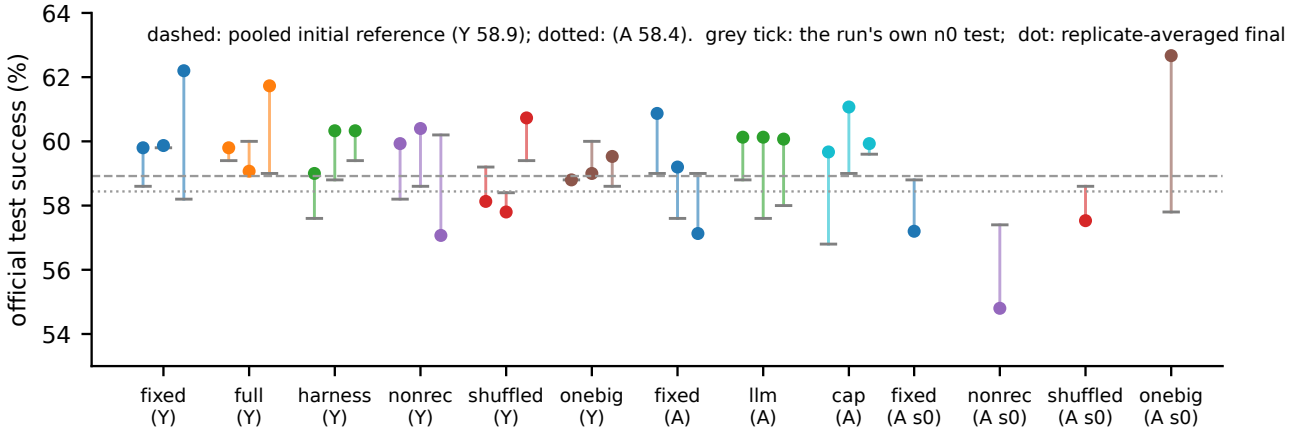


Figure 3. **Every run of the LIBERO-Goal study.** Grey tick: the run's own initial-policy test; dot: the replicate-averaged final of the retained policy; dashed/dotted lines: the pooled initial reference per platform. No gated run ends below its reference beyond noise; the one-shot `onebig` policy of seed 0 is the best single final.

Table 3. **Per-arm finals** (%, mean of the 3-evaluation finals over seeds 1–3; seed-0 runs listed separately). Reference 58.9 (Y) / 58.4 (A).

	`fixed`	`full`	`llm`∗	`nonrec`	`shuf.`	`onebig`
Y, s1–3	60.6	60.2	59.9	59.1	58.9	59.1
A, s1–3	59.1	–	60.1 / 60.2	–	–	–
A, s0	57.2	–	–	54.8	57.5	62.7

∗ Y: `llm_harness`; A: `llm` / `llm_harness_cap`.

task that is never solved (so GRPO gets no signal), the task that is already saturated, and the remaining gate changes that sit inside the noise, and the edits they propose are the ones a practitioner would try. None of them moves success within six rounds: the final retained gates are 62.8 (`fixed`), 63.5 (`full`), 65.0 (`llm_harness`) and 61.7 (`onebig`), all within gate noise. *The object loop reaches the same plateau under every recipe in the space*, so a correct diagnosis of a recipe that already works has nothing to repair.

Self-repair: a recipe that fails. A meta level can be told apart from RL only where RL's recipe fails and a different recipe would not. The study contains one such case, found before any meta-level run: behaviour cloning on the policy's own temperature-1.6 samples with a unit-temperature likelihood collapses when its versions are accepted unconditionally (58.5% → 5.5%). The repair is a one-knob change (the BC temperature, the sampling temperature, or the update operator), and all three knobs are in the harness space. We registered a study (amendments 24 and 26) before any of its runs: start from the collapsing recipe; arms `fixed` (the gate alone), `full` (numeric search) and `llm_harness` (same compute); four seeds; bars S1 = `llm_harness` − `fixed` ≥ +3 with CI above 0, and S2 = `llm_harness` − `full` reported with its CI. The failure case was written down in advance: if S1 fails,

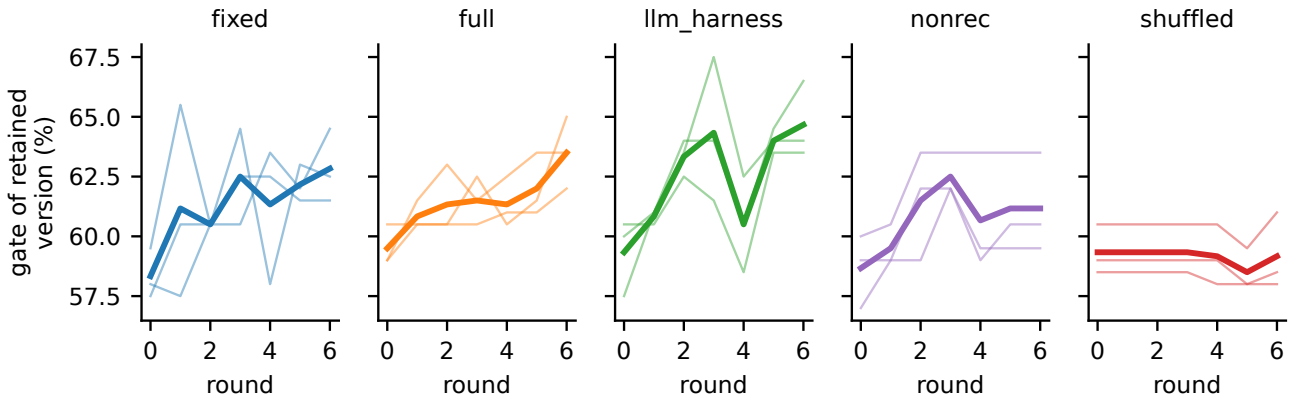


Figure 4. **Gate of the retained version by round** (cloud platform, seeds 1–3 thin, mean bold). The gated arms climb 3–5 points at the gate within six rounds; `shuffled` (verifier removed) stays flat; `nonrec` (practice from the frozen initial policy) stops after round 3. Gate gains overstate test gains by the winner's curse (Tab. 3).

the meta level has no demonstrated effect in this study.

Finding 5 (gate-level, not a claim): the gate alone prevents the collapse, and the LLM diagnoses the failure unprompted. Figure 5 and Tab. 4 give the record. No `fixed` run falls below its start; one seed is held at the initial policy for all six rounds and the other three accept small BC gains (60–64.5%). The LLM's first-round diagnoses on all four seeds name the mismatch between the sampling and likelihood temperatures and the missing failure signal (verbatim in the released logs), and its first proposal on every seed is the same: switch to GRPO on mixed-outcome scenes at a lower temperature. The gate kept that switch on three seeds (gates 64.5–65.5%) and refused it twice on the fourth, where the LLM then repaired BC instead (lower sampling temperature; retained gate 64.5%). Numeric search also finds repairs (a GRPO switch on one seed, the temperature-matched BC likelihood on another). At the gate all arms end within noise of each other (60–65.5%), so S1 and S2 will be decided by the sealed test. We looked for a second failing scenario with six gate-only pilots (four extreme GRPO/BC recipes, both platforms); none degraded under the gate, so no second scenario was registered.

5.4. RQ4: Does the picture transfer to a second domain?

Finding 6: verified feedback gives significant gains, and recursion separates only after the frozen generator is used up. The verified loop improves every model on at least one test (Tab. 5, Fig. 6: CRUXEval-O $+2.6$ for the 1.5B model, LCB-execution $+5.8$ for the 7B model, CIs excluding 0), the permuted-label control does not, and the difference is significant on the trained task family ($+2.7$ $[0.9, 4.5]$). The numeric meta level does not beat the fixed recipe in any three-seed cell. Recursion cannot be detected at four rounds (-0.6) but is significant at eight ($+2.8$ $[1.2, 4.5]$, replicated on the 7B model at $+2.9$ $[0.9, 4.8]$ and $+4.5$ $[1.6, 7.4]$ on LCB-execution). The reason is visible in the records: the frozen generator's practice is as good as the improving model's for about four rounds, after which the non-recursive arm stops accepting gate gains (2 against 8 accepted gains in rounds 5–8) while the recursive arm keeps finding cases at its new frontier. The extra gain is on the trained task family with no extra forgetting on the secondary suites, which is the shape predicted for iterated on-policy learning [12]. It is also why we expect the VLA recursion result ($+1.7$ at six rounds) to be on the same curve, before it separates.

5.5. RQ5: How much of a published VLA-RL number is the test set?

Finding 7: about a fifth of the headline number. LIBERO is tested on 50 fixed initial states per task, and the released SimpleVLA-RL LIBERO-10 checkpoint was trained by RL on exactly those states. We evaluated it, and its own SFT starting point, on the 500 official episodes and on 500 placements freshly drawn by LIBERO's own sampler (the same distribution, not a perturbation): SFT 14.6% official against 20.0% fresh; RL 87.4% against 76.0%. The difference of the two gaps, which cancels any difference in difficulty between the two placement sets, is $+16.8$ points $[+9.8, +23.4]$; a replication on 500 further never-seen placements gives $+16.0$ $[+9.2, +22.4]$ (Fig. 7). The checkpoint is still far better than its start on fresh placements ($+56$ points), so the method works. The point is that about a fifth of the headline number is memorisation of the test states. Every number in this paper is measured on the official states *without* ever training, sampling or selecting

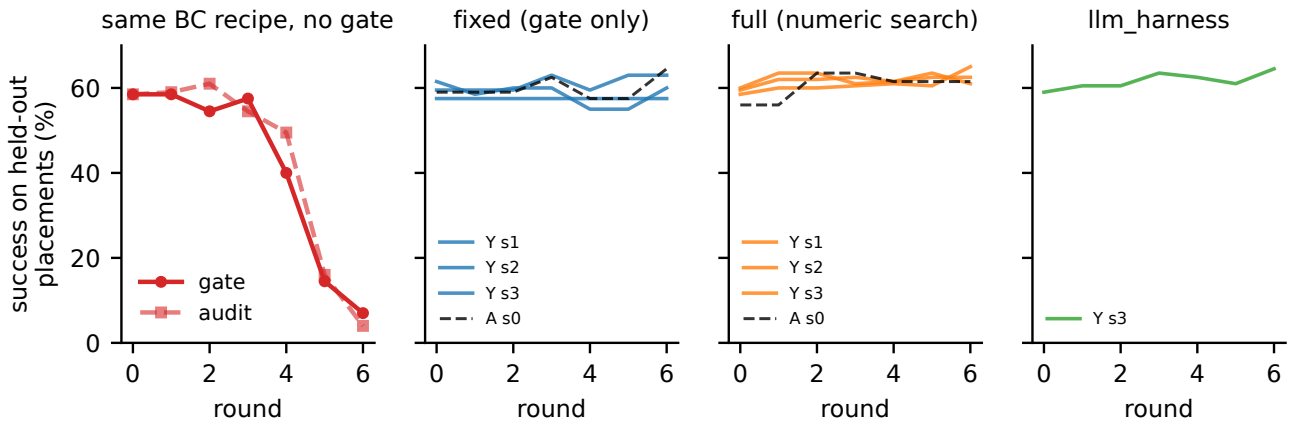


Figure 5. **Self-repair study, gate level.** Left: the starting BC recipe collapses when every version is accepted. Right: under the gate the same recipe never falls below its start (`fixed`); both proposers find repairs that reach 64–65% at the gate; the arms are within gate noise of each other, so the sealed test decides S1 and S2. Runs still in progress are not shown.

Table 4. **Self-repair study, gate-level record** (retained version and its gate, %; what the meta level did). Sealed test results: [to be inserted after the registered read; the study is still sealed].

seed	`fixed` (gate only)	`full` (numeric search)	`llm_harness`
1 (Y)	r6, 60.0; BC unchanged	r5, 62.5; BC unchanged	*running*; LLM switched to GRPO in round 1, kept (65.5 by r3)
2 (Y)	n0 (all 6 refused)	r6, 65.0; switched to GRPO	*running*; LLM switched to GRPO in round 1, kept (64.5, 64.0, 65.5)
3 (Y)	r5, 63.0; BC unchanged	r6, 61.0; BC, rollout-temp. likelihood	r6, 64.5; GRPO refused twice, BC repaired: T 1.6→1.3
0 (A)	r6, 64.5; BC unchanged	r4, 64.0 (rounds 5–6 refused)	*running*; LLM's GRPO child kept in round 2 (65.5 vs 62.5)

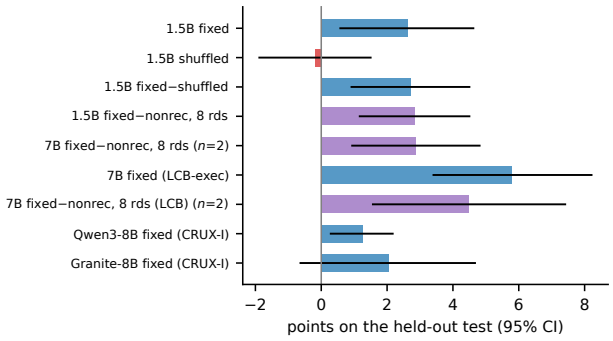


Figure 6. **Code world models.** Verified self-training gains (blue), the permuted-label control (red) and the recursion contrast at eight rounds (purple), with 95% CIs.

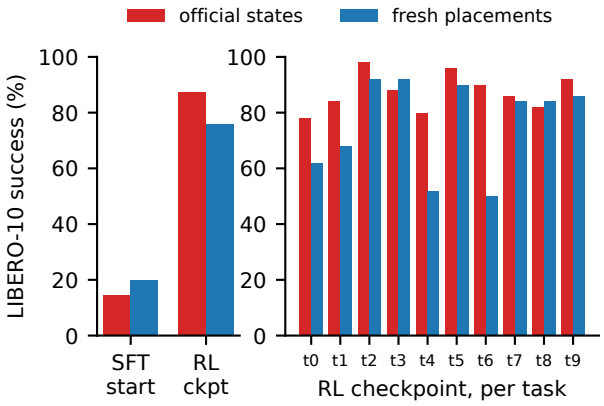


Figure 7. **Train-on-test inflation** (LIBERO-10). Left: official vs. fresh placements for the SFT start and the RL checkpoint trained on the official states. Right: the RL checkpoint per task; the loss is concentrated on tasks 4 and 6.

on them, so our finals should be compared with the "fresh" column of such results.

6. Discussion

What we claim. (1) A gated self-improvement loop for VLA policies is safe at this budget: with fresh paired gates no run degrades on the sealed test, including runs whose recipe collapses without the gate. (2) Its gains on LIBERO-Goal are small, one to two points, and cannot be told apart from one round of plain RL at equal compute, at six and at twelve rounds; recursion is the largest ingredient, just under its bar at six rounds and far over it at twelve, where the loop without recursion degrades. (2b) The gate's protection is

Table 5. **Code world models**, final − initial on the held-out tests (points, seed-averaged paired bootstrap, 95% CI). Four rounds unless noted; "8 rds" = the long-horizon recursion study; † two seeds.

model	comparison	CRUXEval-O	CRUXEval-I	LCB-exec
Qwen2.5-C 1.5B	`fixed` − init	$+2.6\ [0.6, 4.6]$	$+1.2\ [-0.8, 3.3]$	$-1.3\ [-4.4, 1.7]$
	`shuffled` − init	$-0.2\ [-1.9, 1.5]$	$-0.1\ [-1.5, 1.3]$	$-2.4\ [-4.7, 0.0]$
	`fixed` − `shuffled`	$+2.7\ [0.9, 4.5]$	$+1.3\ [-0.5, 3.0]$	$+0.9\ [-1.6, 3.5]$
	`full` − `fixed`	$+0.8\ [-0.7, 2.4]$	$-0.8\ [-2.3, 0.6]$	$+0.7\ [-1.7, 3.1]$
	`fixed` − `nonrec`	$-0.6\ [-2.2, 1.1]$	$+0.3\ [-1.3, 1.8]$	$+0.9\ [-1.3, 3.1]$
	`fixed` − `nonrec`, 8 rds	$+2.8\ [1.2, 4.5]$	$+0.2\ [-1.5, 1.9]$	$-0.5\ [-3.1, 2.1]$
Qwen2.5-C 7B	`fixed` − init	$+1.4\ [-0.2, 3.0]$	$+2.0\ [0.2, 3.8]$	$+5.8\ [3.4, 8.2]$
	`full` − `fixed`	$+0.8\ [-0.5, 2.0]$	$+0.1\ [-1.5, 1.6]$	$+1.1\ [-1.0, 3.1]$
	`fixed` − `nonrec`, 8 rds†	$+2.9\ [0.9, 4.8]$	$0.0\ [-2.1, 2.1]$	$+4.5\ [1.6, 7.4]$
Qwen3-8B	`fixed` − init	$+0.5\ [-0.2, 1.3]$	$+1.3\ [0.3, 2.2]$	$+0.2\ [-0.5, 0.9]$
	`full` − `fixed`†	$+0.7\ [-0.8, 2.2]$	$+2.6\ [0.6, 4.7]$	$+0.5\ [-1.0, 2.1]$
Granite-3.3 8B	`fixed` − init	$+1.4\ [-0.8, 3.7]$	$+2.0\ [-0.6, 4.7]$	$+2.4\ [-1.0, 5.9]$

conditional: it holds for recursive practice at this budget and fails over twelve rounds for practice from a frozen policy. (3) A meta level that edits hyperparameters or harness components does not beat a fixed, tuned recipe, on two studies and two platforms, and the reason is visible in the records: every recipe in the space reaches the same plateau within six rounds. (4) In the code domain the verified loop gains several points and recursion is significant at eight rounds. (5) A published VLA-RL score is inflated by about 17 points by training on the test states.

What we do not claim. We do not claim that recursion lifts a VLA policy at this budget: at twelve rounds it keeps the loop from drifting down, which is a different and smaller thing than the gain the code domain shows, and the verifier's estimate is positive but under its bar. We do not claim that a language-model meta level cannot help: the self-repair study is the test that can show it, and we will report its registered result either way.

For practitioners. Three things in these records can be used directly. A fresh paired gate turns an unsafe self-training recipe into a safe one for the price of one extra evaluation per candidate, and it is the only ingredient whose effect is unambiguous here. Averaged finals and a pooled initial reference are necessary at this scale: a single 500-episode evaluation carries about ± 1.5 points of noise, enough to turn a preliminary pass into a registered fail, which is what happened to our recursion claim. And a language-model meta level is worth its cost only where the recipe can fail; on a tuned recipe its correct diagnoses buy nothing.

Limitations. The VLA results are on one simulated benchmark family with a 7B policy and a few-demonstration start; longer horizons, other policies and real robots are untested here (real-robot data collection is in progress). The gate is noisy (sd 3.4 points at 200 episodes), which both limits the meta level and makes the loop conservative; a larger gate costs proportionally more simulation. Evaluation noise on the official test is about ± 1.5 points per evaluation, which is why finals are averaged and why bars below $+2$ would not be readable at this budget. Some runs were continued after hardware or wall-time interruptions, in each case from a completed round with the same plan; the amendments disclose every such event. Finally, preregistration constrains the analysis but not the design: we chose the seed recipe and the harness space, and a different space might leave more for a meta level to find.

7. Conclusion

We set out to learn which part of a self-improvement loop helps a robot policy, and to find out under rules that could not bend to the answer. The answer on LIBERO-Goal is modest and, we think, useful. The gate is the part that matters: it turns a recipe that would collapse into one that never regresses. Recursion and the verifier each add about a point and a half at six rounds, less than the bars we set; at twelve rounds recursion is what keeps the loop from drifting below its start, and it is still not better than one round of plain RL. A meta level that edits the recipe adds nothing while the recipe works, for a reason the records make visible, and the one situation in which it could matter, a broken recipe, is under a sealed test. In a second domain the same loop gains more and recursion pays once the frozen generator runs out of useful practice. Along the way we measured that a published VLA-RL score owes about a fifth of itself to the benchmark's fixed test states. We hope the protocol—sealed tests, fresh placements, averaged finals,

public amendments, one read—outlives the particular numbers.

References

[1] Samuel Albanie, James Thewlis, Sébastien Ehrhardt, and João F. Henriques. On the preregistration of machine learning experiments. *NeurIPS Workshop on Preregistration in ML*, 2021. 3

[2] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In *ICML*, 2024. 2, 3

[3] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv:2501.12948*, 2025. 2, 3

[4] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. In *ICML*, 2024. 5

[5] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (ReST) for language modeling. *arXiv:2308.08998*, 2023. 2, 3

[6] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *ICLR*, 2025. 3

[7] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report. *arXiv:2409.12186*, 2024. 5

[8] IBM Granite Team. Granite 3.x language models. https://github.com/ibm-granite/granite-3.0-language-models, 2024. 5

[9] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv:2403.07974*, 2024. 5

[10] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model. In *CoRL*, 2024. 3

[11] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv:2502.19645*, 2025. 3, 4

[12] Kim et al. Coverage improvement and fast convergence of on-policy preference learning. *arXiv:2601.08421*, 2026. 3, 8

[13] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaoyang Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. SimpleVLA-RL: Scaling VLA training via reinforcement learning. *arXiv:2509.09674*, 2025. 2, 3, 4

[14] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023. 2, 4

[15] Jijia Liu, Feng Gao, Bingwen Wei, Xinlei Chen, Qingmin Liao, Yi Wu, Chao Yu, and Yu Wang. What can RL bring to VLA generalization? an empirical study. *arXiv:2505.19789*, 2025. 2, 3

[16] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv:2408.06292*, 2024. 3

[17] Guanxing Lu, Wenkai Guo, Chaoyi Zhang, Yue Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv:2505.18719*, 2025. 3

[18] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *PNAS*, 115(11):2600–2606, 2018. 3

[19] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*, 2025. 3

[20] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024. 2, 3

[21] Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. In *TMLR*, 2024. 3

[22] An Yang et al. Qwen3 technical report. *arXiv:2505.09388*, 2025. 5

[23] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. In *ICML*, 2024. 2, 3

[24] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *NeurIPS*, 2022. 3

[25] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv:2505.03335*, 2025. 2, 3